

## SKILL\_WEEK-09

**Task 1:** To Change directories, validate paths, update the working directory, handle errors, maintain the previous directory, and test navigation.

```
UW PICO 5.09

    {
        perror("chdir failed");
    }
}
else
{
    char temp[PATH_SIZE];

    if (getcwd(temp, sizeof(temp)) == NULL)
    {
        perror("getcwd failed");
        continue;
    }

    if (chdir(path) == 0)
    {
        strcpy(previous, temp);

        getcwd(current, sizeof(current));
        printf("Current Directory: %s\n", current);
    }
    else
    {
        perror("Invalid directory");
    }
}

return 0;
}
```

**Fig 1:** C program for directory navigation, path validation, and maintaining the previous directory

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill9_task1
Initial Directory: /Users/amrutha/OSSP

Enter directory path (or '-' for previous, 'exit' to stop): ..
Current Directory: /Users/amrutha

Enter directory path (or '-' for previous, 'exit' to stop):
```

**Fig 2:** Successful directory change using a valid relative path

```
Enter directory path (or '-' for previous, 'exit' to stop): -
Changed to previous directory: /Users/amrutha/OSSP
```

**Fig 3:** Returning to the previous working directory using stored directory state

```
Enter directory path (or '-' for previous, 'exit' to stop): /not/a/real/path
Invalid directory: No such file or directory
```

**Fig 4:** Handling an invalid directory path during navigation



**Fig 5:** Flowchart for directory navigation and previous-directory management

## Result:

Directory navigation was implemented successfully using `chdir()` and `getcwd()`. Valid paths were handled correctly, invalid paths produced appropriate errors, the current working directory was updated, and the previous directory was maintained for backward navigation.

**Task 2:** Identify built-ins, create a dispatch table, execute in-process commands, handle invalid commands, maintain state, and test dispatch logic.

```

UW PICO 5.09
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 256

int builtin_pwd(char *arg)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("Current Directory: %s\n", cwd);
    }
    else
    {
        perror("getcwd failed");
    }

    return 1;
}

int builtin_cd(char *arg)
{
    if (arg == NULL)
    {
        printf("cd: directory required\n");
        return 1;
    }

    int builtin_cd(char *arg)
    {
        if (arg == NULL)
        {
            printf("cd: directory required\n");
            return 1;
        }

        if (chdir(arg) == 0)
        {
            printf("Directory changed successfully.\n");
        }
        else
        {
            perror("cd failed");
        }

        return 1;
    }

    int builtin_help(char *arg)
    {
        printf("Available Built-in Commands:\n");
        printf("pwd      - Display current directory\n");
        printf("cd <path> - Change directory\n");
        printf("help     - Display help\n");
        printf("exit     - Exit mini shell\n");

        return 1;
    }

    int builtin_exit(char *arg)

```

**Fig 6:** C program implementing built-in commands using a dispatch table and function pointers

```

[amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill9_task2.c -o bin/skill9_task2
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill9_task2

mini-shell> pwd
Current Directory: /Users/amrutha/OSSP

```

**Fig 7:** Executing the built-in pwd command through the dispatch table

```

mini-shell> help
Available Built-in Commands:
pwd      - Display current directory
cd <path> - Change directory
help     - Display help
exit     - Exit mini shell

```

**Fig 8:** Displaying supported built-in commands through the dispatch table

```

mini-shell> cd ..
Directory changed successfully.

mini-shell> pwd
Current Directory: /Users/amrutha

```

**Fig 9:** Maintaining shell state after executing the built-in cd command

```
mini-shell> hello
Invalid built-in command: hello
```

**Fig 10:** Handling an invalid command during dispatch-table lookup

```
mini-shell> exit
Exiting mini shell...
```

**Fig 11:** Exiting the mini shell using the built-in exit command



**Fig 12:** Flowchart for executing built-in commands using a dispatch table

## Result:

Built-in commands were identified and executed successfully using a dispatch table and function pointers. Commands such as pwd, cd, help, and exit were handled within the same process, shell state was maintained, and invalid commands were detected correctly.